Preventing Wrong-Target Agent Actions in Spatial Interfaces with Executable Interaction Contracts

ANONYMOUS AUTHOR(S)

Embodied agents in three-dimensional interfaces can return a fluent response even when the selected object, responsible agent, invoked operation, and reported evidence do not agree. Such failures are difficult to detect when scene bindings, agent roles, handoffs, and backend endpoints are wired in separate artifacts. We present an executable interaction contract for spatial multi-agent interfaces, realized as FunctionalMLDS across a Unity client and a conversational-agent backend. The contract pins one model version and requires each admitted request to resolve one spatial target, one declared provider, one capability use, and one concrete runtime action. The backend re-resolves these relations rather than trusting client fields; stale, ambiguous, unreachable, or multiply bound requests fail before a response-model call and session mutation whenever the violation is request-decidable. Response-dependent violations are rejected transactionally.

We use a complementary two-study evaluation. A deterministic system study examines coverage, migration non-regression, runtime enforcement, added cross-layer expressiveness, and structural cost in three authored environments. All 93 scene assets participate in complete declared interaction paths, the direct-wiring and contract representations agree on all 459 route classifications under a fixed shared policy, and the runtime admits all 298 local or direct routes while rejecting all 161 transitive or unreachable routes before the response stub without retained mutation. A complementary remote, task-based user study with 65 completed sessions evaluates the visible target, handoff, and responsibility cues. Participants generally reported clear answer–target fit and handoff transitions; among 49 who reported a handoff, 45 found the final agent clear. However, only 37/65 selected the intended object-and-topic responsibility rule. The contribution is therefore a fail-closed control and evidence boundary with initial descriptive evidence about its intelligibility, not a causal claim that it improves usability over an alternative interface.

CCS Concepts: • **Human-centered computing** → **Intelligent user interfaces**; *Interactive systems and tools*; • **Computing methodologies** → *Intelligent agents.*

Additional Key Words and Phrases: intelligent user interfaces, embodied agents, spatial grounding, interaction contracts, runtime assurance, model-driven engineering, Unity

ACM Reference Format:

Anonymous Author(s). 2026. Preventing Wrong-Target Agent Actions in Spatial Interfaces with Executable Interaction Contracts. 1, 1 (August 2026), 27 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

A visitor enters a virtual exhibition, points to a product display, and asks an embodied agent, “What is this?” The environment might contain multiple agents (such as a receptionist, a product specialist, or a career advisor), each with unique responsibilities and capabilities. Generating a fluent answer does not guarantee that the interaction was correct: the system must also track the targeted object, identify the responsible agent, verify that the required capability is

Manuscript submitted to ACM

explicitly declared for that agent, execute the intended action, and ensure the resulting response remains anchored to the same object.

This execution chain easily breaks because its components are spread across different architectural layers: a 3D engine (e.g., Unity) manages scene objects, colliders, and user state; configuration files specify agent roles; orchestration logic routes requests; and backend services call models or tools, frequently logging nothing beyond request success. Consequently, individual components may appear valid locally while the overall interaction fails—for instance, the answer might refer to a different object, originate from an unauthorized agent, or stem from an undeclared operation. Although our prototype uses Unity, this cross-layer mismatch is representative of similar multi-tiered architectures.

In intelligent user interfaces, successful network transport or model invocation does not equal interaction validity. Before text generation starts, the system must answer four deterministic questions: which object was targeted, which agent owns responsibility for it, which capability is assigned to that agent and request, and which specific backend action implements this capability. Following generation, the system must verify that the output remains aligned with the selected object and responsible agent.

We present an *executable interaction contract*, FunctionalMLDS (Functional Meta-Language-defined Structure), extending the MLDS (Meta-Language-defined Structure) artifact [15]. This contract maintains alignment between target, provider, capability, action, and evidence under a fixed model version, while rejecting and localizing incomplete, outdated, ambiguous, unreachable, or conflicting execution chains.

Three key requirements, grounded in human–AI interaction guidelines [1, 36, 38], drive this design:

Stable spatial reference. The focused object remains the active target throughout request construction, and the output is presented only when returned identifiers match that target.

Visible unresolved states. Conditions like *no target* or *ambiguous target* explicitly prevent deictic queries instead of defaulting to a fluent yet ungrounded response.

Capability-aware, inspectable routing. Agent selection depends on modeled roles and capabilities; the chosen provider, handoff step, capability, and action are preserved as structured evidence to inform UI cues or user explanations.

While these mechanisms establish a technical control boundary, their utility in intelligent interfaces depends on whether users can comprehend the interaction flow. The contract activates only after a target object is chosen; it neither infers unstated intent nor guarantees the factual accuracy of generated text. However, it guarantees that the chosen object is reliably linked to a declared provider, capability, and action, making parts of this linkage visible to the user.

Consequently, we evaluate our system from two perspectives: a technical benchmark measuring contract coverage, non-regression, runtime enforcement, fault localization, and overhead cost; alongside a user study examining whether participants correctly identify the target object, observe handoffs, recognize the responding agent, and understand the system's delegation of responsibilities based on contract-driven visual cues. This empirical evaluation focuses specifically on the implemented interaction, measuring task clarity, predictability, guidance, and usability, rather than long-term trust or general cross-domain applicability.

We address four central research questions:

RQ1—Coverage and non-regression. Can multi-agent 3D environments be adapted to end-to-end target–provider–capability–action chains while maintaining existing ownership rules and routing logic under a fixed policy?

RQ2—Fail-closed enforcement. Does the implemented communication path accept only valid local and single-hop requests, retain target and provider evidence, and cleanly reject stale, ambiguous, unreachable, or conflicting requests?

RQ3—Added assurance and cost. What cross-layer inconsistencies can the contract catch and isolate beyond direct artifacts, and what are the associated representation and performance overheads?

RQ4—User comprehension. How accurately do users perceive the chosen target, the object–answer mapping, handoffs, and agent responsibilities, and how predictable and supportive do they find the interaction?

This work provides four main contributions:

(1) an interaction-centered failure taxonomy and executable contract binding spatial selections to a single responsible provider, capability, action, and evidence trace under a pinned model version;
(2) a fail-closed system architecture spanning a Unity 3D client and multi-agent backend featuring authoritative server-side resolution, staged validation, strict one-hop handoff rules, and transactional response verification;
(3) a reproducible, API-free technical evaluation across three 3D environments, including chain audits, non-regression comparisons with direct wiring, routing probes, fault injection, and structural cost analysis; and
(4) a user study evaluating target clarity, answer–target alignment, handoff perception, agent role recognition, predictability, and user guidance.

Our technical evaluation indicates that the contract maintains baseline system behavior, strictly enforces one-hop policies, and detects cross-layer discrepancies that native artifacts miss. The empirical user study complements these findings: while users readily grasped object selection, target–answer grounding, and handoff events, agent selection rationales and functional boundaries were less intuitive. This demonstrates that while a contract can reliably enforce backend responsibilities, additional interface cues are necessary for full user clarity.

The complete prototype, model artifacts, evaluation scripts, raw data, study materials, and reproduction guidelines are made available in the anonymous repository. We do not claim that formal metamodels alone grant intelligence to interfaces; rather, explicit relations constrain what systems interpret and execute, reducing errors like wrong-target references or undeclared actions into predictable, auditable bounds.

Section 2 outlines related research on embodied agents, spatial reference, conversational XR, model-driven engineering, and runtime verification. Section 3 details the proposed contract and its formal guarantees; Section 4 describes its implementation across Unity, the backend, and evidence tracing. Sections 5 and 6 present technical benchmarks and user study findings. Section 7 explores system trade-offs, costs, and communication boundaries, while Sections 8 and 9 outline limitations and conclusions. Supplementary materials are included in the appendix.

2 Related Work and Positioning

2.1 Embodied agents and spatial reference

Embodied conversational agents combine language with a visible body, gesture, gaze, and shared space [10]. Museum-guide systems show why the location and social role of an agent matter to interaction [3, 21], while recent generative-agent and multi-agent work adds memory, role specialization, and language-mediated coordination [25, 33]. These systems motivate the multi-agent setting considered here, but a role or dialogue policy does not by itself bind an utterance to one Unity entity and one executable operation.

Situated-language research addresses the upstream reference problem. Approaches combine plan recognition with referring expressions [16], map directions into perceived environments [20], maintain symbolic anchors for discourse

[23], and learn perceptually grounded word meanings [19]. MM-Conv extends this line with synchronized speech, motion, gaze, and scene context for ambiguous reference in dynamic 3D dialogue [13]. Our contract does not compete with these resolvers: it accepts a candidate selection and checks whether the modeled interface permits that target to reach a specific provider and action. A learned or multimodal resolver could therefore replace the current desktop ray without changing the downstream admission questions.

2.2 Conversational XR and model-based interface engineering

Language models increasingly author or operate immersive environments. LLMR uses prompting, scene understanding, planning, execution, and self-debugging to edit mixed-reality worlds [12]. Tell-XR supports conversational end-user development of event–condition–action automations [9]; CUIfy packages LLM-powered conversational agents for XR [4]; and AgentHands studies gestures for spatially grounded agent conversations [29]. These systems establish that conversational generation, agent embodiment, and XR interaction are valuable and feasible. The issue studied here is narrower: after a world and its agents exist, can the interface prevent a selected object, responsible agent, backend action, and returned evidence from silently diverging?

Model-based interface development has long separated domain, task, abstract interface, and concrete implementation concerns [7, 35]. UsiXML and MARIA support multiple development paths and abstraction levels [28, 34], while OpenUIDL addresses runtime omni-channel interfaces [32]. Model-driven work for immersive applications likewise generates AR applications from domain models [8] or structures VR requirements around roles, scenes, actions, states, and validation [18]. FunctionalMLDS is not a general UI description language. It specializes the runtime link among spatial entities, embodied-agent responsibility, permitted capabilities, concrete operations, and observable assertions.

The closest lineage is MLDS. Fischer et al. define a Meta-Language-defined Structure as a machine-readable artifact governed by a domain-specific language specification [15]. Prior work already uses this idea for natural-language-to-Unity materialization and optional placement refinement [6], and modular MLDS instructions add profiles, dependencies, versions, and gatekeeper validation for evolving toolchains [5]. Consequently, this paper does not claim novelty for MLDS-to-Unity generation, placement, or modular language specification. Its contribution begins at interaction time: the same typed relations used during materialization remain active as an admission and evidence contract.

2.3 Human–AI intelligibility, control, contracts, and provenance

Human–AI interaction guidelines emphasize communicating system capability and status, making uncertainty visible, enabling correction, and supporting recovery [1, 36, 38]. Lim and Dey organize intelligibility around recurring user questions such as *What*, *Why*, *Why not*, *What if*, and *How to* [27]; Liao et al. adapt this question-driven perspective to explainable-AI design [26]. We use these established questions to organize which contract relation can be exposed at each interaction gate, rather than introducing a competing explanation taxonomy. Because explanations do not automatically improve appropriate reliance or joint performance [2], the contract supplies inspectable state while the user study separately evaluates whether the implemented cues support target, handoff, and responsibility comprehension.

Design by Contract separates preconditions, postconditions, and invariants [31]; runtime verification evaluates specified properties against execution observations [24]. We adopt this enforcement intuition without claiming formal program verification. The relevant obligations cross implementation layers: a request must use a pinned model, a unique target, a responsible and reachable provider, and an exact action binding; a response must preserve those identifiers. W3C PROV shows how entities, activities, and agents can support exchangeable provenance [22], and failure-attribution work demonstrates that output-only traces are insufficient for multi-agent diagnosis [11]. Our evidence is intentionally

Table 1. Positioning by primary unit and assurance locus. The distinctions state the focus of this paper; they do not imply that a cited system lacks mechanisms outside its stated contribution.

Work	Primary represented unit	Main interface/runtime emphasis	Distinction of this work
LLMR [12] and Tell-XR [9]	World edits or event–condition–action automations	Conversational generation and end-user authoring	We constrain an already materialized request through target, provider, capability, and action.
Spatial grounding [16, 23]	Referring expression, perception, and candidate referent	Inferring what a person refers to	We begin after candidate selection and test whether the selected entity has a permitted executable path.
Model-based UI engineering [7, 34, 35]	Tasks, domain concepts, and abstract/concrete interfaces	Transformation across abstraction and deployment levels	We specialize the runtime boundary to spatial entities, agents, capabilities, actions, and evidence.
MLDS lineage [5, 15]	Machine-readable artifacts and modular language specifications	Generation, validation, profiles, and toolchain evolution	We keep one typed artifact active during request admission and response presentation.
This work	Pinned target–provider–capability–action–assertion contract	Fail-closed spatial request admission and relation-level evidence	The contribution is the composed boundary and its executable, corpus-bounded evaluation.

smaller than a full reasoning trace: it records contract identifiers and observable outcomes, not private chain-of-thought or an explanation of semantic answer quality.

Requirements traceability is relevant because a link is valuable only when it supports reasoning about change and failure [17, 30]. The evaluation therefore does not count links alone. It tests whether valid cases resolve end to end, whether controlled mutations are localized, whether runtime requests are admitted or rejected at the intended boundary, and what representational cost the links introduce.

2.4 Positioning

Table 1 distinguishes the unit constrained by representative work. Conversational-XR systems primarily generate or control worlds; reference-resolution systems infer a target; model-based UI approaches connect abstract and concrete interfaces; and multi-agent systems coordinate roles. The present contract starts from a selected candidate and constrains the specific path that may cross the Unity–backend boundary.

3 Interaction Contract

Software contracts usually guard technical boundaries. In an intelligent interface, the same decisions also determine what the system treats as selected, which agent may respond, and how users recover from a blocked request. Our running scenario takes place in an explorable 3D trade-fair booth for a cheese manufacturer. Visitors can move through distinct exhibition areas, inspect products and production exhibits, and address several embodied agents with different responsibilities. A visitor who is currently speaking with the Reception Agent selects a refrigerated cheese display counter (`refrigerated_counter1`) and asks, “What cheese is displayed here?” The counter is assigned to the Product Specialist Agent. The contract must therefore preserve the selected counter, resolve that provider, authorize the declared direct handoff, invoke the grounded-answer capability, and present the response only if the returned evidence still names the same target and provider.

Table 2. Cross-layer failures, the user questions they raise, and the current prototype response. The last column is one presentation mapping, not a requirement of the formal contract.

Failure	Contract condition and decision	User question	Prototype response
Target identity	Entity or source identifiers are unknown, stale, or non-unique; reject before routing.	*What object did the system select?*	Invalidate the target highlight and state that the object is unavailable; do not generate an answer.
Target ambiguity	No unique referent exists; reject instead of selecting a candidate silently.	*What does the current selection refer to?*	Keep the unresolved state visible and request clarification or a new selection.
Responsibility drift	The active agent is neither the unique provider nor one permitted handoff away; reject after responsibility resolution.	*Why this agent, and why not the active one?*	Expose the responsible provider and any direct handoff; otherwise give a scoped refusal.
Action shortcut	No unique scenario–capability–binding–action path exists; reject before a response-model call.	*Why can this request not proceed?*	Show neutral capability-unavailable guidance without invoking the response model.
Evidence mismatch	Returned target, provider, binding, or action differs from the admitted path; reject presentation and restore provisional state.	*What happened to the request?*	Suppress the mismatching answer while retaining the pending target and a correction point.
Undeclared handoff	The proposed agent transition is not allowed; roll back before commit.	*Why was the delegation blocked?*	Cancel the transition and keep the interaction in a recoverable state.

3.1 Failure model and interface implications

A fluent answer can conceal an unknown target, an unauthorized provider, an unavailable operation, or evidence for another object. Table 2 links each failure to a deterministic decision, a user question, and the prototype response.

We interpret the questions in Table 2 through Lim and Dey's intelligibility types and later question-driven XAI work [26, 27]. The questions are an interpretive lens, not model elements or a second taxonomy. Target, provider, capability, and evidence are typed states from which the interface can answer *What*, *Why*, and *Why not* questions about the interaction.

The prototype uses progressive disclosure: a spatial highlight marks the current target, a lightweight provider or handoff indication appears when responsibility changes, and concise guidance appears for unresolved targets or unavailable capabilities. Internal identifiers, route reasons, bindings, and assertion records remain developer-facing. This mapping operationalizes system-status visibility and recoverability in human–AI interaction guidance [1], but it is only one possible presentation. The same contract could support textual or spoken explanations, a persistent responsibility panel, or an explicit clarification dialogue. We neither compare these alternatives nor infer calibrated trust from the current cues: explanations can increase acceptance without improving appropriate reliance [2]. We therefore evaluate the implemented mapping as support for comprehension and control.

3.2 Contract objects and executable path

A contract-grounded interaction is represented by

$$\mathcal{I} = \langle m, s, t, p, u, c, b, a, q, o \rangle,$$

where m is the pinned model and hash, s the scenario step, t the target, p the responsible provider, u the context-specific capability use, c the reusable capability, b the runtime binding, a the concrete action, q the assertions, and o the observation evidence. Read in execution order, m, s fix the context; t, p identify what was selected and who may act; u, c, b, a declare what may run; and q, o check whether the result still agrees with the admitted request. In the running scenario, t is `refrigerated_counter1`, p is the Product Specialist Agent, and the remaining elements declare and verify the grounded answer about the selected cheese display.

The required declaration path is

$$s \to u \to c \to b \to a.$$

A scenario step cannot invoke a directly. Its capability use must name the trusted target and provider and resolve exactly one capability, binding, and action. A generic `POST /chat` call therefore cannot bypass the declaration that the Product Specialist Agent may answer product questions about this counter.

The provider is resolved independently of display names. For target t, the resolver selects the most specific non-empty responsibility tier:

$$R(t) = \begin{cases} R_{\text{asset}}(t), & \text{if non-empty,} \\ R_{\text{group}}(t), & \text{otherwise if non-empty,} \\ R_{\text{zone}}(t), & \text{otherwise.} \end{cases}$$

Exactly one provider at the selected tier is required; no provider or several providers make the request inadmissible. This is a deterministic specificity policy, not a model of whom visitors intuitively expect to answer: an asset-level assignment overrides broader group and zone defaults. For `refrigerated_counter1`, the policy selects the Product Specialist Agent. If the Reception Agent is active, one declared handoff may reach that provider. The interaction-design consequence is deliberately modest but important: when the resolved provider differs from the active agent, the change must not be silent. The contract returns the provider and route reason; the interface decides how to communicate them.

3.3 Admission and response conditions

A deictic request is admitted only if all of the following conditions hold:

(1) **Environment identity.** The session is pinned to m, and its hash matches the current project and stored contract fingerprint.
(2) **Target identity.** Entity and source identifiers resolve to the same unique target t, with a resolved, non-ambiguous selection state.
(3) **Provider authorization.** $R(t)$ yields exactly one provider p, which is either active or one permitted direct handoff away.
(4) **Executable capability.** One capability use u resolves to exactly one capability c, binding b, and action a.
(5) **Schema conformance.** The request satisfies the declared action schema.

These gates connect directly to the failure model: Conditions 1–2 cover target identity and ambiguity, Condition 3 covers responsibility drift, and Conditions 4–5 prevent action shortcuts. Evidence mismatch and undeclared handoff are checked after generation. In the running scenario, preflight confirms the pinned booth and refrigerated counter, resolves the Product Specialist Agent, permits the Reception Agent's direct handoff, and selects the grounded-answer action. A failed condition yields a relation-specific rejection rather than model-generated fallback text. After generation, the

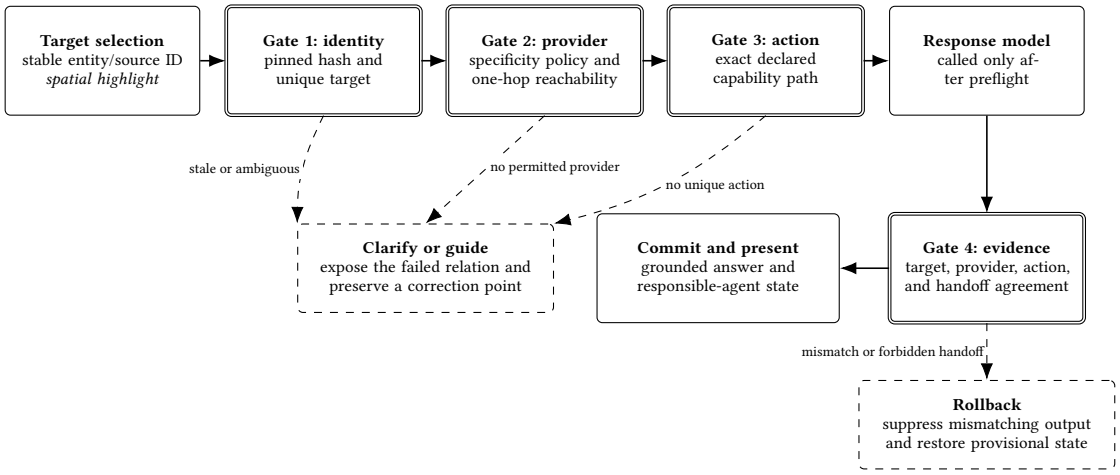

Fig. 1. Fail-closed interaction boundary. Identity, provider, and action checks precede the response-model call. Each preflight failure exposes a relation-specific state for clarification or guidance. After generation, evidence agreement determines whether the answer is committed or rolled back. The shown highlight, provider/handoff indication, and guidance states are the prototype mapping rather than requirements of the contract.

returned target, provider, route, binding, and action are checked against the admitted tuple and assertions q. Agreement is recorded in o and permits commit; a mismatch or undeclared handoff restores the snapshot and blocks presentation.

3.4 Guarantee boundary

The guarantee has three parts:

- **System consistency.** An admitted request has one trusted target, one declared provider reachable by at most one handoff, one capability-to-action path, and matching response evidence. State is committed only when those relations agree.
- **Interface support.** The same relations provide stable state for target feedback, provider and handoff disclosure, capability-specific guidance, and recovery. The contract makes these signals possible but does not prescribe their modality, wording, timing, or visual form.
- **Not guaranteed.** The contract does not establish that the visitor intended the selected collider, that generated content is factually correct, that the chosen cue mapping is optimal, or that users will understand it or calibrate trust appropriately.

The interaction claim is empirical: the contract supplies target, provider, handoff, capability, and rejection state, while the user study examines whether the current cues support target-selection clarity, answer–target fit, handoff understanding, final-agent clarity, perceived responsibility, predictability, guidance, and overall ease.

4 From Model to Interface

Section 3 defined the relations that must remain consistent. This section follows the same boundary from the implementation side and focuses on what changes for a visitor: a retained target highlight, an observable handoff, capability-specific guidance, and either a grounded answer or a recoverable rejection. We continue the cheese-booth scenario in which the Reception Agent hands a question about `refrigerated_counter1` to the Product Specialist Agent. For readability, we

call the canonical FunctionalMLDS V2 instance the *room contract*, its trace-map/runtime-context projection the *runtime map*, and the model hash or contract fingerprint the *content hash*; Appendix Table 7 maps these shorter terms to the repository terminology.

4.1 Model relations that drive interface state

A Meta-Language-defined Structure (MLDS) is a machine-readable artifact whose vocabulary and structural rules are defined by an explicit meta-language specification [15]. Each room-specific FunctionalMLDS document is such an instance. Its role at interaction time is to provide one authoritative chain from the selected spatial entity, through the responsible agent and permitted capability, to the runtime action and the evidence required before an answer may be shown.

FunctionalMLDS reuses selected UML and EAST-ADL use-case, requirements, and verification concepts as its scenario foundation [14]; it does not claim complete UML or EAST-ADL conformance. The runtime-relevant extension adds stable spatial entities, agent responsibilities and handoffs, situated capability uses, concrete actions, and expected evidence. Appendix Table 8 gives the compact element-and-relation map needed to follow the migration and coverage claims in RQ1.

The distinction between a reusable capability and one situated use of it is central. "Answer grounded questions" is a capability; "the Product Specialist Agent answers about `refrigerated_counter1` in this step" is a capability use. The latter connects a visible target and responsible agent to one executable path, allowing the interface to retain the counter, show a provider change, or report that the requested operation is unavailable.

A conventional dialogue manager or finite-state machine could reproduce the nominal behavior of this single walkthrough. The difference appears when scene bindings, agent roles, handoffs, and backend actions evolve in separate artifacts. Consider a drift case in which Unity still selects the refrigerated counter, the role configuration assigns it to the Product Specialist Agent, but a stale backend route invokes a generic Reception Agent action. A state machine can prevent that failure only if it also imports and synchronizes the same target, provider, action, and evidence relations. FunctionalMLDS makes that synchronization an explicit, typed, and versioned contract. The user-perceivable benefit is not additional interface chrome: the inconsistent turn is blocked, the selection is retained, and guidance can identify the failed relation instead of presenting a plausible response from the wrong execution path. We do not compare implementation effort against a state machine, so our claim is consistency and fault localization across layers, not that the formal representation is always the simplest authoring mechanism.

4.2 Prototype cue rationale and empirical status

The contract requires distinguishable target, responsibility, capability, and outcome states, but it does not prescribe their visual form. The prototype uses a minimal mapping guided by four design constraints: spatial locality, event salience, limited persistent clutter, and actionable recovery. The target is highlighted in the scene because the referent is spatial and the cue can remain co-located with it while the visitor composes a question. A transient provider or handoff status appears only when responsibility changes, making the transition visible without occupying permanent scene space. Text is reserved for blocked or unresolved states because recovery requires an actionable instruction. Exact identifiers, bindings, and assertion records remain developer-facing. This mapping operationalizes system-status visibility and recoverability rather than exposing the complete contract [1].

These choices were not selected through a prior comparative design study, pilot study, or documented heuristic inspection. They are fixed prototype decisions and therefore a design hypothesis, not an optimized cue set. Textual or

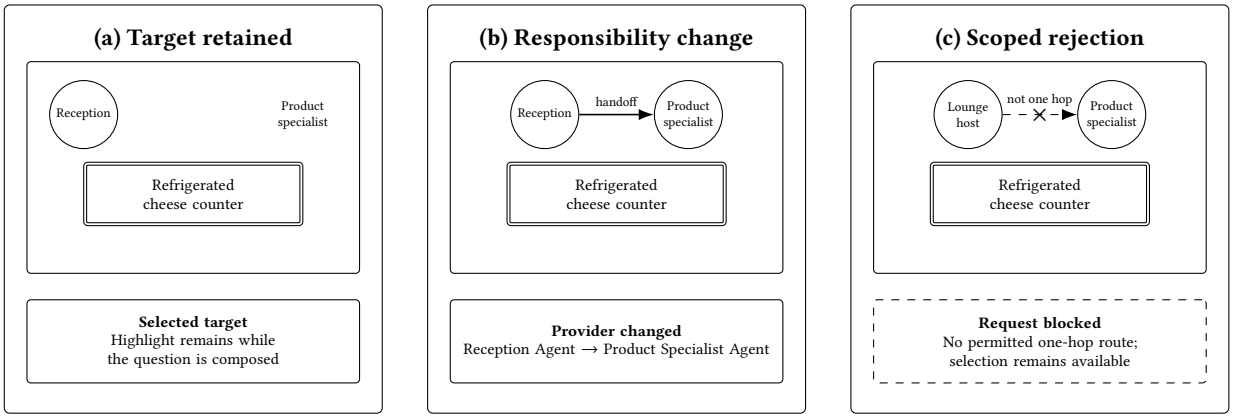

Fig. 2. Illustrative interface-state mockup for the running scenario. The panels show the target, handoff, and rejection states produced by the same contract boundary as Figure 1. This is an explanatory reconstruction rather than a pixel-accurate screenshot; the labels paraphrase runtime state and are not prescribed by the formal model.

spoken explanations, a persistent responsibility panel, and an explicit clarification dialogue remain plausible alternatives. The user study is the first empirical assessment of the implemented mapping and examines target clarity, handoff understanding, perceived responsibility, guidance, and overall ease; it does not compare those alternatives or establish that the chosen cues optimally calibrate trust.

Figure 2 makes the mapping concrete. It presents the same target–provider–action–evidence boundary as Figure 1, but at the user-visible moments produced by the implementation rather than as a second gate diagram.

4.3 When consistency is checked

The four validation mechanisms are not four sequential operations paid on every conversational turn. Before a room opens, serialization and model checks reject malformed fields, broken references, non-unique source identifiers, and contradictory relations. Before an interaction is enabled, the executable-profile check requires one complete target–provider–capability–action path. Per turn, runtime checks bind the request and response to the pinned room contract. Appendix Table 11 lists the detailed invariants and failure boundaries.

Room objects use stable `id` and `sourceId` values rather than display labels or engine object names. Agents have corresponding source identifiers and typed references to their capabilities, responsibilities, grounded objects, and handoff targets. The room contract remains authoritative, while the client and backend receive only the runtime subset they need. A visible label can consequently change without silently changing the target or provider addressed by the interaction.

During authoring, structured room data and frozen semantic artifacts are transformed into entities, agent roles, scenarios, capability uses, runtime bindings, assertions, and validation cases. Language models may propose semantics, role labels, handoffs, and knowledge, but deterministic stages assign runtime identifiers and routes. Atomic publication and content hashes keep the room contract and its client and backend projections together: a stale client, mismatching backend, or partially regenerated room fails during setup instead of entering an interaction with inconsistent responsibilities.

4.4 3D-client selection and visible state

The evaluated client is implemented in Unity, but the binding principle is engine-independent: every selectable scene object must map to exactly one stable model entity. A camera ray selects a registered collider, the client marks it with a spatial highlight, and that target remains active while the visitor composes the request. The states `none`, `resolved`, and `ambiguous` are explicit. Only a resolved state enables a deictic request; the other states remain visible and lead to guidance rather than an inferred target.

A scene-object component binds each relevant collider to an exact entity and source identifier. At startup, the registry rejects duplicate or conflicting bindings and missing target-specific contexts. The request carries the content hash, exact target identifiers, bounded ray and candidate data, input modality, active agent, and utterance. The backend repeats the authoritative identity check rather than relying on the client alone.

The main walkthrough and Figure 2 remain on the refrigerated counter. Appendix Figure 3 is a separate frozen implementation capture of `brand_panel3`; it verifies the same collider-to-entity mechanism and is not a second running example.

The client bridge uses the selected entity and server-resolved provider to locate one target-specific scenario context. It accepts only the scenario, capability, binding, and action identifiers confirmed by the backend; zero or several exact contexts leave the interaction unresolved.

4.5 Authoritative routing, execution, and recovery

At session setup, the backend loads the room contract and runtime map, builds the executable context, and pins the content hash. For each request, it rejects malformed, stale, ambiguous, or conflicting spatial payloads and derives the selected asset's group and zone from the room contract rather than from client-provided responsibility fields.

Responsibility resolution applies the asset–group–zone precedence from Section 3. The active agent may answer locally or hand off once along a declared edge. A resolved provider change is returned as explicit state for the handoff cue; the absence of a permitted provider yields relation-specific guidance. The backend then selects one exact runtime action from the trusted scenario, provider, and target, so a generic endpoint match is insufficient.

Before generation, the backend snapshots mutable session state. Failures that can be decided from the request stop before the response model and leave the conversation unchanged. Violations visible only after generation, such as an undeclared handoff or mismatching returned identifier, restore the snapshot and suppress the output. Model-call success alone is therefore insufficient for presentation.

Responsiveness boundary. Only target/provider/action lookup, schema checks, evidence comparison, and snapshot bookkeeping lie on the per-turn contract path; authoring and room validation occur before the conversation. The current evaluation nevertheless does not measure end-to-end interaction time. Its only timing evidence is an in-memory structural workload of 6.15–10.06 ms for the contract representation; it excludes file parsing, Unity, networking, and model inference. These values are not conversational latency, and we make no claim about perceived responsiveness. Full turn time and users' sensitivity to delay remain to be evaluated for live spatial interaction.

4.6 Evidence, feedback, and identifier strictness

The runtime trace serves two audiences. For presentation, the client compares the returned target, provider, route, binding, and action with its pending selection and the admitted contract path. Matching evidence permits the answer to be shown while retaining the responsible-agent state; a mismatch produces rollback or corrective guidance. For

diagnosis, runtime events also retain the content hash, scenario and step, capability use, capability, assertion references, and observation verdicts. Missing observations remain `inconclusive` rather than silently passing.

Strictness applies to structured contract metadata, not to the wording of the generated answer. The model may describe the counter in different natural language, but the returned target, provider, binding, and action identifiers must match the admitted path. A known alias could be normalized before this comparison; an unknown or different identifier is rejected even when the prose appears plausible. This deliberately favors referential and authorization consistency over fuzzy identifier recovery. The current prototype does not implement approximate matching.

The visitor interface exposes the selected or unresolved target, makes a provider change observable through the evaluated handoff state, and shows concise status or guidance when the request cannot proceed. Exact route reasons, bindings, identifiers, and assertion records remain in developer logs. The user study evaluates this lightweight mapping; richer explanations and persistent responsibility panels are not implemented or compared.

4.7 Scope and scaling boundary

The contribution is not object anchoring or dialogue state by itself. Reference-resolution and conversational-XR systems can maintain a local candidate referent or dialogue context [9, 12, 23]. The room contract starts from that selected candidate and additionally constrains who may respond, whether a handoff is permitted, which concrete action may run, and which identifiers must return before presentation. Section 2 positions this boundary against those systems in more detail.

The evaluated corpus contains 93 routable assets and four to six agents per room. The runtime map avoids scanning the full authoring model for each turn, and the interface exposes only the current target, provider transition, and status rather than all modeled relations. Thus the visible cue count does not grow with the number of objects in the room. We have not evaluated larger agent graphs, multi-hop execution, crowded selections, or whether users remain able to understand responsibilities in substantially denser scenes.

4.8 End-to-end walkthrough

The complete path is concrete. The visitor selects the refrigerated cheese counter and keeps its highlight while asking, “What cheese is displayed here?” The client submits the counter’s exact identifiers and the active Reception Agent. The backend resolves the Product Specialist Agent, verifies the declared direct handoff, and invokes only the grounded-answer action declared for that target and provider. The response is shown only when its evidence still names the same counter, provider, route, and action; otherwise the provisional state is restored and guidance is displayed. Starting the same request from the Lounge Host requires two handoff edges through the Reception Agent, so it is rejected before generation and the conversation remains unchanged.

5 Evaluation Method

5.1 Complementary two-study design

We evaluate the contract at two boundaries. *Study T* addresses RQ1–RQ3 with deterministic software evidence: whether the authored relations are complete, survive migration, reject invalid requests, and add enforceable invariants at measurable structural cost. *Study U* addresses RQ4 with a remote, unmoderated, task-based evaluation of the implemented interface: whether visitors can follow its target, provider, and handoff cues. The studies are complementary.

Study T cannot establish human comprehension; Study U cannot validate the internal contract or show superiority over another architecture.

5.2 Study T: contract coverage and enforcement

The technical corpus comprises the three authored repository cases: a career-fair room, a classroom, and the food/trade-fair brand room (93 routable assets and 15 agents). A fresh native V2 instance is regenerated from each frozen version-0.5 case. An asset is complete only when every asset-targeting `ScenarioStep`/`CapabilityUse` pair resolves one coherent provider, capability, binding, action, assertion set, and covering validation case.

Separately parsed direct-wiring and V2 adapters expose ownership candidates and handoff edges to one fixed policy: asset ownership precedes group and zone ownership, and several owners at the highest active tier are ambiguous. We compare the owner of every asset and the route from every modeled starting agent. Because one shared function classifies both normalized views, agreement tests migration parity rather than policy correctness. Derived copies exercise group fallback, zone fallback, and ambiguity because every natural asset has an explicit asset owner.

Runtime enforcement converts all 459 object–start-agent pairs into backend requests. Local and direct routes must preserve target, provider, capability, binding, and action evidence; transitive and unreachable routes must stop before the deterministic response stub without retained mutation. A targeted suite adds one valid and 16 invalid requests, and three Unity Editor smokes execute the generated client bridge and evidence checks. Common and contract-only mutations test detection, localization, and false positives. An alternating 40-repetition in-memory benchmark measures structural adapter cost; it excludes file I/O, generation, Unity, networking, and model latency. All technical runs are API-free.

5.3 Study U: browser-based interaction study

Study U evaluates the user-visible realization, not the metamodel in the abstract. Participants used the WebXR-capable brand-room prototype through a standard browser; the current analysis cut contains desktop/laptop and tablet/smartphone sessions but no headset session. A short questionnaire guided two tasks in a fixed order. First, participants selected a product exhibit and asked the Product Specialist a grounded question, exercising target visibility and answer–target fit. Second, they asked the Reception Agent a product question designed to invoke the declared direct handoff to the Product Specialist, exercising handoff awareness and final-agent clarity. Immediate post-task questions reduced recall demands.

Both tasks deliberately exercise successful target and direct-handoff paths. The study therefore evaluates whether normal contract-grounded interaction is intelligible; it does not experimentally induce target ambiguity, an unavailable capability, an unreachable provider, or response rollback. Error explanation was rated only when participants happened to encounter such a state. The fixed task order kept the unmoderated procedure short and uniform, but prevents separating task from order effects.

The instrument uses five-point agreement items, a 0–10 overall score, one multiple-choice comprehension check, device and experience questions, and optional formative comments. The comprehension proportion is the primary human-study outcome; the custom ratings are secondary. Its distractors encode nearest-agent, first-contacted-agent, and unclear mental models, while the introduction describes the multi-agent setting without disclosing the intended object-and-topic rule. With no comparison condition, Study U supplies descriptive evidence about the current cues, not evidence that FunctionalMLDS improves usability over direct wiring or another interface.

5.4 Participants, ethics, sample adequacy, and analysis

Participants were recruited by convenience sampling; the questionnaire export does not encode a recruitment source for individual sessions. The analysis cut of 17 August 2026 contains 65 unique completed sessions: 45 on desktop or laptop browsers and 20 on tablets or smartphones. Participants were 18–54 years old (41 aged 18–24, 20 aged 25–34, and 4 aged 45–54); 44 reported computing, engineering, or design/HCI backgrounds, 11 business, and 10 other fields. The sample characterizes the current prototype audience and is not population-representative.

Participation was voluntary. Before beginning, participants received study information and provided informed consent; they could discontinue without penalty. The questionnaire collected no names or direct contact identifiers and stored responses under pseudonymous session identifiers. Results are reported only in aggregate.

Because Study U is single-condition and descriptive, a group-difference power calculation would be inappropriate. We instead assess estimation precision. With $n = 65$, the largest two-sided 95% Wilson interval half-width for a binary proportion is approximately 0.12 at $p = .5$ [37]. This supports broad item-level and majority-versus-ambiguity claims, but not small subgroup differences or population estimates. Later responses will form a new dated data cut rather than being silently appended.

For Study T we report exact corpus counts and descriptive timing; inferential intervals would imply a sampling model the exhaustive probes do not have. For Study U we report item medians and interquartile ranges, positive ratings (4 or 5), and categorical counts; the comprehension proportion receives a 95% Wilson interval. Handoff ratings are restricted strictly to participants with `q5_handoff_observed=yes`; "no" and "unsure" responses are excluded even if downstream fields contain numeric values. Error-feedback ratings include only numeric responses to `q9`; "not experienced" is excluded. We do not combine the custom items into an unvalidated omnibus scale or perform confirmatory subgroup tests. Optional comments are used formatively.

The questionnaire export records perceived handoffs but no linked per-session backend event. RQ4 therefore concerns reported transition awareness and understanding; Study T independently establishes that the modeled handoff path is executable. The dated export, analysis code, technical inputs, mutations, and raw probe outcomes define the reproducibility boundary.

6 Results

Table 3 summarizes the principal outcomes. The values are exact for the fixed three-case corpus and deterministic test paths. They are not participant counts or dialogue-quality scores.

6.1 RQ1: complete migration with routing non-regression

All 93 scene assets occurred in at least one declared interaction chain, and all 93 satisfied the asset-completeness definition. The chain audit examined 186 asset-targeting `ScenarioStep`/`CapabilityUse` pairs: 54 in the career-fair case, 72 in the classroom, and 60 in the brand room. Every pair resolved one capability and provider, agreed with its scenario performer and target, reached an executable binding and action, resolved all resulting assertions, and was covered by an appropriate validation case. No asset-chain error was reported. The two pairs per asset encode answer and handoff paths; they are not two executed conversations.

Natural-corpus ownership was unique for 93/93 assets. Every decision occurred at the explicit asset tier; no natural object exercised group or zone fallback, and none was ambiguous or unassigned. In the separate derived priority suite,

Table 3. Main measured results and their interpretation boundary.

Check	Observed result	Supported interpretation
Complete declarations	93/93 scene assets and 186/186 asset-targeting step–capability-use pairs complete	The three regenerated models contain provider-coherent paths through binding, action, assertion, and validation coverage.
Migration non-regression	93/93 owner decisions and 459/459 route classifications agree	V2 preserves the direct artifacts under the fixed shared resolver; this is not an independent routing oracle.
Runtime admission	298/298 local/direct routes admitted; 161/161 transitive/unreachable routes rejected before the stub	The implemented one-hop policy is enforced and preserves target and provider evidence without retained mutation on rejection.
Common injected faults	Both adapters detected and localized 9/9; 0/3 false positives	Parity for the three shared fault types.
Contract-only faults	Strict provider contract detected and localized 9/9; 0/3 false positives; canonical model validator detected 3/9	The executable profile adds typed cross-layer invariants absent from the direct artifacts.
Unity execution	3/3 batch entry points passed; one multi-scenario smoke loaded 31 contexts and 73 actions and bound 30/30 scene objects exactly	The evaluated desktop runtime components execute the generated contract; no headset or user outcome is established.

both adapters passed all nine decision cases: three group fallbacks, three zone fallbacks, and three ambiguity rejections. Across both adapters this is 18/18 expected outcomes, kept separate from natural-corpus frequencies.

The 459 object-by-start-agent routes comprised 93 local-owner cases, 205 direct handoffs, 143 transitive-only graph paths, and 18 unreachable pairs. Local-or-direct one-hop coverage was therefore 298/459 (64.9%), while offline graph reachability was 441/459 (96.1%). The latter does not imply that the online runtime traverses several handoffs within one turn.

The direct-wiring and fresh-V2 adapters selected the same owner for all 93 objects and the same route class for all 459 pairs. They also remained aligned after all nine common mutations. Because both normalized views are classified by one shared policy function, this result establishes migration non-regression and serialization parity, not that V2 discovered a more correct routing policy.

6.2 RQ2: fail-closed enforcement

The executable runtime sweep passed all 459 probes. It admitted 298/298 local or direct routes and made exactly 298 deterministic-stub calls. Every admitted response preserved the expected source object, target entity, and routed provider and returned non-empty capability-use, capability, binding, and action identifiers that agreed across the checked response, grounding, routing, and runtime-action fields.

The runtime rejected 161/161 transitive or unreachable routes before the stub and without retained session mutation. This set contains the 143 transitive-only and 18 unreachable pairs. The result demonstrates the current one-hop admission boundary, not task failure in principle: a system with a different declared policy could permit multi-hop execution, but this implementation does not.

The targeted spatial suite accepted its valid deictic request and rejected all 16 invalid variants without retained mutation. Fifteen variants were request-decidable and stopped before the stub. The remaining variant used a structured response that proposed an undeclared handoff; one stub call was necessary to reveal the destination, after which

Table 4. Contract-only mutations. "Not represented" means that the direct artifacts contain no equivalent typed relation; it is not counted as a baseline miss.

Mutation (once per scene)	Violated invariant	Direct artifacts	Executable provider contract
Remove `CapabilityUse.provider`	An executable capability occurrence has exactly one provider agreeing with the performing agent and target responsibility.	Not represented	3/3 detected and localized
Duplicate `Agent.sourceAgentId`	Runtime-facing source-agent identity is unique.	No typed equivalent	3/3 detected and localized
Assign a domain capability to the runtime orchestrator	Domain behavior is provided by the modeled domain agent, while the orchestrator realizes infrastructure actions.	Not represented	3/3 detected and localized

provisional history was rolled back. Rejections included stale model hashes, unknown or contradictory identifiers, ambiguous selections, invalid agents, excessive payloads, and non-finite or out-of-range spatial values.

All three Unity Editor batch entry points returned success. Together they exercised scenario progression, dispatch, five assertion types, valid and deliberately broken evidence, selection-state blocking, and ambiguous registries. The multi-scenario Steinpilz smoke loaded 31 contexts and 73 actions and established unique exact bindings for 30/30 scene objects without UUID-suffix fallback. These observations confirm that the client-side bridge and evidence components execute the materialized contract; they do not measure frame rate, end-to-end latency, or human understanding.

6.3 RQ3: added invariant scope and cost

On the common mutation denominator, both adapters accepted all three unmodified case copies, for 0/3 false positives per adapter. Both detected and localized 9/9 changes: one missing owner, one duplicate owner, and one dangling handoff per scene. The result is deliberately neutral. Direct wiring can enforce these three relationships when supplied with a validator and the same policy.

The representation patches expose a mixed trade-off. Across the nine common mutations, the contract representation changed 9 top-level artifacts compared with 15 for direct wiring, but changed 18 stored references compared with 15. These counts describe the scripted patches and do not establish that a developer would work faster or make fewer mistakes.

The contract-only suite provides the positive evidence for additional expressiveness. Table 4 reports the three typed mutation classes. The canonical model validator detected the duplicated source-agent identifier in each case (3/9 overall) but intentionally permits some optional relations for compatibility. The executable pipeline agent–provider contract detected and localized all 9/9 mutations with 0/3 false positives on valid copies. The direct artifacts have no typed capability-use provider or domain-capability-to-runtime-orchestrator relation, so there is no equivalent baseline outcome to score.

Table 5 reports the current benchmark snapshot. The fresh-V2 adapter required 5.89–7.61 times the direct-wiring median for the measured in-memory workload. Absolute medians ranged from 6.15 to 10.06 ms for V2 and 1.04 to 1.32 ms for direct wiring. Outputs remained deterministic across all repetitions. These values exclude file parsing, generation, Unity, networking, and model inference and must not be read as end-user response latency.

Table 5. Local structural workload over 40 measured repetitions per adapter and case. Values are median [Q1, Q3] in milliseconds.

Case	Direct	Fresh V2	Ratio
Career fair	1.322 [1.167, 1.516]	10.057 [7.784, 11.573]	7.61
Classroom	1.246 [0.772, 1.609]	9.439 [5.681, 13.336]	7.58
Brand room	1.044 [0.992, 1.694]	6.151 [4.951, 7.309]	5.89

Taken together, RQ3 does not show that an explicit contract is universally better than direct artifacts. It shows a more specific trade: shared ownership and handoff faults can be validated in either representation, while the contract makes additional provider, identity, capability, binding, and evidence relations explicit and enforceable at measurable structural cost.

6.4 RQ4: comprehension of the implemented interaction

The dated user-study cut contains 65 completed sessions. Target selection was rated positively (4 or 5) by 45/65 participants (69.2%; median 4, IQR 3–5), and 50/65 rated task-level answer–target fit positively (76.9%; median 5, IQR 4–5). In the final ratings, 55/65 found the guidance helpful (84.6%), 47/65 rated the overall interaction easy (72.3%), and the 0–10 overall score had a median of 8 (IQR 7–9).

Forty-nine participants answered "yes" when asked whether they observed the intended handoff; eight were unsure and eight answered "no." Restricting the handoff ratings strictly to those 49, 40 rated the transition understandable (81.6%; mean 4.41, median 5, IQR 4–5) and 45 found the final responding agent clear (91.8%; mean 4.67, median 5, IQR 5–5). Numeric downstream ratings attached to "no" or "unsure" responses were not included. These values measure perceived handoff visibility. The export contains no linked per-session backend event, so individual log–report agreement cannot be tested.

Responsibility rationale was less consistently understood. The general responsibility item was positive for 42/65 participants (64.6%; median 4, IQR 3–4), but only 37/65 selected the intended rule that provider choice depends on the selected object and question topic (56.9%; 95% Wilson CI [44.8%, 68.2%]). Ten selected the first-contacted agent, eight the nearest agent, and ten reported that the rule was unclear. Likewise, only 30/65 rated the division among several specialized agents as helpful (46.2%; median 3, IQR 3–4). Thus the handoff event and final agent were more visible than the underlying responsibility rule.

Only 22 participants encountered a state for which they supplied a numeric error-explanation rating; 15/22 rated it positively. Because neither task deliberately induced ambiguity, a blocked capability, an unreachable provider, or rollback, this opportunistic subset does not establish user comprehension of fail-closed behavior. Optional comments primarily identified navigation and input friction, unclear or overlapping agent roles, and answer-presentation or content-quality issues. These observations guide interface refinement but do not alter the technical contract results.

7 Discussion

7.1 Why the contract is an intelligent-interface contribution

The contract does not generate the linguistic content that makes an agent appear intelligent. It determines whether the interface may use and present that content for a particular spatial interaction. This distinction matters because a wrong-target answer can be both fluent and locally plausible. By binding the pending selection to one provider and action before generation, the system turns a silent semantic-looking failure into a deterministic admission decision.

The user-facing consequence is limited but concrete. A deictic request cannot proceed from a missing or ambiguous selection; a returned answer cannot be presented as grounded when its target or provider disagrees with the pending interaction. The current implementation exposes the detailed route and binding primarily to developers, not visitors. A future interface could use the same evidence to explain, for example, "This object is handled by the Reception Agent" or to offer a clarification choice. Whether such explanations improve understanding, reliance, or recovery remains an empirical question.

7.2 What the direct-wiring comparison establishes

The comparison is intentionally less dramatic than an architecture competition. For explicit asset ownership, handoff edges, and a fixed priority rule, direct artifacts can produce the same routing decisions and detect the same three common fault types. This is an important result: introducing FunctionalMLDS did not change the behavior already encoded in the source artifacts, but neither did the migration automatically improve it.

The added value appears where the interaction crosses artifact boundaries. Direct wiring has no typed occurrence connecting a scenario target to one provider, capability, binding, action, and assertion. Consequently, relations such as "this capability use has no provider" or "this domain capability is provided by the runtime orchestrator" are not representable as baseline invariants. The strict provider contract detected all such injected faults. This supports a scoped adoption argument: the model is useful when a system needs lifecycle-wide target–provider–action evidence, not merely an owner lookup table.

7.3 When the added structure is justified

A static room with one chatbot and a small, stable set of objects may not justify more than direct configuration and tests. The contract becomes more plausible when several agents divide responsibility, the same action kind has many target-specific bindings, scenes are regenerated, or developers need to trace a failure across Unity and backend artifacts. Even then, its cost is not free. The current representation stores more references for the common patches and its normalized adapter is substantially slower than direct wiring, although the measured workload remains small relative to a typical model call.

The benchmark does not measure authoring effort. Fewer changed top-level files could reduce search and coordination, while more explicit references could increase editing burden. A developer study must test this trade rather than inferring it from patch counts.

7.4 Interaction design opportunities

The present boundary supports several interaction patterns that are not yet implemented as evaluated visitor experiences:

Clarification. When several candidates or owners exist, the interface can present the alternatives instead of only blocking the request.

Transparent handoff. Before delegating, the active agent can name the responsible provider and reason, then let the visitor confirm.

Correction. A visitor or author can replace the selected target or report that the proposed responsibility is wrong; the model relation can then be revised through a typed transaction.

Abstention with location. A rejection can distinguish "I cannot identify the object," "no agent is responsible," and "the responsible agent is not reachable" rather than presenting one generic failure.

These patterns use evidence the system already produces, but their wording, timing, and cognitive cost require design and evaluation. They are stronger future directions than adding more internal trace fields without an interface that uses them.

7.5 Priority evidence needed beyond this study

Two additions would most improve external validity. First, a fourth independently authored, frozen scene should contain natural group- and zone-level responsibility, at least one unassigned object, and one genuine ambiguity. Resolver and validator code should be frozen before the case is revealed. This would test transfer without deriving every interesting condition synthetically.

Second, expected routes should be supplied by an independent oracle or a separately implemented resolver rather than the same shared classification function. A declarative table reviewed before execution would be sufficient for a modest held-out case. These technical additions address the strongest circularity concern without making unsupported human claims.

A developer task study is the most relevant human follow-up if ethics review, participants, and sufficient time are available. Participants could diagnose and repair target, provider, and handoff faults using direct artifacts or the contract representation. Completion rate, time, inspected artifacts, incorrect repairs, and explanation accuracy would test the proposed diagnostic benefit. A small convenience study using only subjective ratings would add less evidence than a well-controlled held-out technical evaluation.

7.6 Authorization and policy composition

"Permitted" in this paper means declared by the interaction model. The contract neither authenticates a visitor nor grants access to protected data or actuators. A deployment can first resolve the exact target–provider–action tuple and then submit it to an external authorization policy. Capability lifecycle governance can separately decide which implementation version is active. The current prototype implements neither identity management nor resource authorization and should not be presented as a security boundary by itself.

7.7 Privacy, bias, and accountability

A spatial-agent request can reveal utterances, selected objects, locations, and interaction histories. The backend receives the utterance, stable target identifiers, ray metadata, modality, and bounded history. Persistent runtime events omit the utterance text in the evaluated path, but the prototype lacks visitor-facing consent, retention, export, and deletion controls. A deployment should minimize stored spatial data and separate technical entity identifiers from personal identities.

Explicit agent responsibility can make organizational assumptions easier to inspect, but it can also formalize biased or inappropriate authority. A validator can establish that one provider is declared consistently; it cannot establish that the provider assignment is fair, representative, or safe. Model review, knowledge provenance, refusal behavior, and domain safeguards remain necessary.

8 Limitations and Threats to Validity

The technical evaluation establishes behavior for three purposively authored development cases, not frequencies in a population of rooms, domains, or authors. Every natural asset has an explicit asset-level owner, so group and

zone fallback and ambiguity are exercised only in derived copies. No independently authored or held-out scene tests generalization.

The non-regression comparison is controlled rather than independent. Both representations originate from shared semantic artifacts and are classified by one routing function after separate parsing; the direct-wiring projection also supplies expected routes for the runtime sweep. Agreement therefore detects migration divergence but cannot validate the shared policy against an external oracle. Binding identifiers are checked for agreement within one pinned model, not against independently authored labels.

Fault injection covers three common and three contract-only mutation types. Detection and localization do not imply complete fault coverage or repair, and scripted deltas are not developer effort. The V2 assembler, validators, and materializer share a codebase, so their agreement is not independent standards conformance. Runtime tests use a deterministic structured stub, isolating admission, evidence, and rollback but not answer correctness or naturalness. Timing covers only in-memory adapter work and excludes the end-to-end path.

Study U is an unmoderated, single-condition convenience study of one brand room. The current cut contains 45 desktop/laptop and 20 tablet/smartphone sessions, but no headset session. Device heterogeneity, self-selection, and the young, technically weighted sample limit generalization. With 65 sessions, binary proportions have at worst an approximately 12-percentage-point 95% Wilson half-width; this supports broad descriptive patterns, not fine subgroup or population claims. Without a baseline, observed comprehension cannot be causally attributed to FunctionalMLDS rather than the interface and tasks.

The two user tasks were presented in a fixed order and both targeted successful paths: one grounded object answer and one permitted direct handoff. Study U therefore does not directly test whether users understand the central fail-closed states in Table 2, including ambiguity, unavailable capabilities, unreachable providers, or rollback. Only 22 participants happened to rate an error explanation, so that result is opportunistic and exploratory. A controlled failure-comprehension task is needed before claiming that the “clarify or guide” and rollback behavior is intelligible to users.

The questionnaire records perceived handoffs but no linked per-session backend handoff events, preventing individual log–report agreement. The custom items are not a validated psychometric scale, and the study does not evaluate long-term use, trust calibration, workload, immersion, or developer diagnosis. It tests only whether the current cues support target, answer, handoff, and responsibility comprehension.

The project remains static within a session. Drift is detected, but there is no live migration for objects created, deleted, merged, or rebound during a conversation. Online routing permits at most one direct handoff per request; transitive reachability is an offline diagnostic. Concurrent writes and simultaneous visitors were not tested. Finally, the selected EAST-ADL subset is a traceability foundation; full conformance has not been independently assessed.

9 Conclusion

We presented an executable interaction contract that keeps a selected spatial entity, responsible embodied agent, declared capability, concrete backend action, and returned evidence within one pinned model version. The client makes unresolved selection explicit, while the backend re-resolves identity, responsibility, route, and action before generation and rolls back response-dependent violations.

Across three authored environments, all 93 assets and 186 target-specific declaration pairs were complete. Migration preserved all 93 owner decisions and 459 route classifications under a shared policy. The runtime admitted all 298 local or direct routes with matching evidence and rejected all 161 transitive or unreachable routes before the response stub

without retained mutation. A strict provider contract additionally detected and localized all nine injected cross-layer faults that rely on relations absent from the direct artifacts.

The complementary user study shows why technical enforcement and interface intelligibility must be evaluated separately. Participants generally reported clear target–answer fit and visible handoffs, but only 37/65 identified the intended object-and-topic responsibility rule, and the usefulness of several specialized agents received mixed ratings. These results provide initial evidence for successful-path intelligibility, not for comprehension of blocked or rolled-back interactions. FunctionalMLDS provides the evaluated fail-closed control and evidence boundary, while the interface still needs clearer responsibility and failure communication.

GenAI Usage Disclosure

The frozen case-study inputs include eight successful structured-output calls to `gpt-4.1-mini-2025-04-14` at temperature 0.2: two scene-semantics artifacts, three agent-role and handoff artifacts, and three sets of local knowledge entries. A remaining scene-semantics artifact was recovered deterministically without a model call. Prompts, returned JSON, model identifiers, token metadata, and validation records are retained with the case artifacts. The authors reviewed and froze these outputs as study inputs; no generative model supplied evaluation labels or served as a semantic judge. Model assembly, routing probes, fault injection, measurements, and reported tables are deterministic and API-free.

A coding assistant supported prototype hardening, test and analysis code, literature-search support, and drafting and editing. The authors inspected and revised the resulting code and prose, verified citations and generated evidence, and remain responsible for all claims.

A Detailed Model, Contract, and Evaluation Tables

A.1 Running-scenario identifiers

Table 6 spells out the exact objects used for the `refrigerated_counter1` scenario. Display labels are shortened in the prose, while execution uses these stable identifiers.

Table 6. Exact contract objects for the Steinpilz `refrigerated_counter1` interaction.

Element	Exact instance	Runtime meaning
Target	`refrigerated_counter1` → `ENT-ASSET-REFRIGERATED_COUNTER1`; group `ENT-GROUP-PRODUCT_COUNTERS`; zone `ENT-ZONE-PRODUCT_DISPLAY_ZONE`	Collider observation and trusted model referent
Use case and scenario	`UC-STEINPILZ_BRAND_ROOM-INTERACT-REFRIGERATED_COUNTER1`; `SC-STEINPILZ_BRAND_ROOM-INTERACT-REFRIGERATED_COUNTER1-MAIN`	User-facing interaction and executable flow
Answer step	`STEP-STEINPILZ_BRAND_ROOM-INTERACT-REFRIGERATED_COUNTER1-ANSWER`	Scenario occurrence requiring one capability use
Capability use	`CU-STEINPILZ_BRAND_ROOM-INTERACT-REFRIGERATED_COUNTER1-ANSWER-ROOM-GROUNDED-QUESTION`; provider `ENT-AGENT-PRODUCT_SPECIALIST_AGENT`	Explicit target tuple and provider
Capability	`CAP-STEINPILZ_BRAND_ROOM-ANSWER-ROOM-GROUNDED-QUESTION`	Operation declared for the Product Specialist Agent
Binding and action	`RB-STEINPILZ_BRAND_ROOM-ANSWER-ROOM-GROUNDED-QUESTION`; `RA-STEINPILZ_BRAND_ROOM-BACKEND-CHAT`	Exact realization at `POST /chat`
Assertion and case	`SA-STEINPILZ_BRAND_ROOM-ANSWER-GROUNDED`; `VC-STEINPILZ_BRAND_ROOM-INTERACT-REFRIGERATED_COUNTER1`	Declared identifier agreement and validation scope

A.2 Runtime terminology

Table 7 maps the shorter terms used in Section 4 to the exact repository and model terminology.

Table 7. Reader-facing runtime terms and their exact artifact counterparts.

Paper term	Repository or model term	Meaning
Room contract	FunctionalMLDS V2 instance; `functionalmlds.v2.instance.json`	Authoritative room-specific model containing interaction, execution, and evidence relations.
Runtime map	Trace map and runtime context	Executable subset used by the client and backend to resolve one target-specific interaction path.
Content hash	Model hash and contract fingerprint	Digest that binds the room contract, generated projections, and active session to the same model version.
Situated capability	`CapabilityUse`	One occurrence that links a scenario step, target, provider, and reusable capability.

A.3 FunctionalMLDS element and relation map

Table 8 complements the interaction-focused view in Figure 2. It records the selected model elements and references used to construct the complete `target–provider–capability–action` paths evaluated in RQ1; it is a compact map rather than a full serialization schema.

Table 8. Selected FunctionalMLDS elements and relations underlying the runtime projection.

Elements	Key relation	Role in the running scenario
`DynamicFunctionalModel`; `RequirementsModel`; `UseCaseDefinitionSet` → `UseCaseDefinition` → `UseCase`	One room-specific root links the selected UML/EAST-ADL requirements and use-case foundation to interaction, execution, and validation elements.	Fixes the authoritative cheese-booth model and scopes the visitor goal represented by the counter interaction.
`UseCaseScenarioSpecification` → `Scenario` → `ScenarioStep`	A use case is refined into ordered steps that identify the performer and the interaction occurrence.	Represents selection of `refrigerated_counter1`, the question, and the grounded-answer step.
`Actor`; `Entity`; `Agent` ⊆ `Entity`	Stable entity/source identifiers ground scene objects; agents additionally reference capabilities, asset/group/zone responsibilities, and permitted handoffs.	Identifies the counter, the active Reception Agent, and the responsible Product Specialist Agent.
`CapabilityUse` → `Capability`	A situated capability use names its scenario step, provider, and targets and resolves one reusable capability.	Binds the Product Specialist Agent and selected counter to the grounded-answer capability.
`RuntimePlatform`; `RuntimeBinding` → `RuntimeAction`	The declared capability is realized by one platform-specific binding and concrete endpoint, tool, or event.	Resolves the accepted interaction to the exact backend-chat action rather than a generic chat dispatch.
`Effect` → `Assertion`; runtime observation; `VerificationValidation/VVCase`	Assertions define expected results, observations record runtime evidence, and validation cases cover the relevant binding and assertions.	Requires returned target, provider, route, and action identifiers to agree before the answer is presented.

A.4 Separate implementation capture

Figure 3 records the exact collider-to-entity binding used by the Unity client. It depicts `brand_panel3`, because it is a frozen implementation capture rather than the running cheese-counter walkthrough; both objects use the same exact entity/source-ID mechanism.

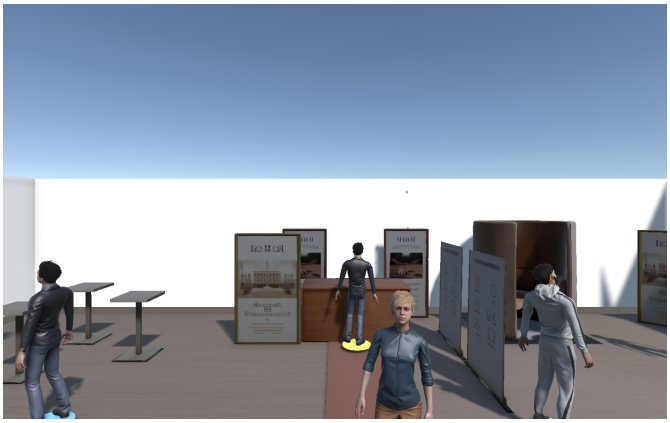

Fig. 3. Separate Unity desktop implementation capture. The selected `brand_panel3` collider is bound to its stable model entity; the main text uses `refrigerated_counter1` as the running scenario.

A.5 Per-case structural results

Table 9. Complete interaction paths. "Pairs" are asset-targeting `ScenarioStep`/`CapabilityUse` pairs.

Case	Assets	Complete	Pairs	Complete
Career fair	27	27	54	54
Classroom	36	36	72	72
Brand room	30	30	60	60
Total	93	93	186	186

Table 10. Offline structural routes from every modeled starting agent. "Transitive" and "reachable" are graph properties, not one-turn online execution.

Case	Probes	Local	Direct	Transitive	Unreachable	Reachable
Career fair	135	27	53	55	0	135
Classroom	144	36	81	9	18	126
Brand room	180	30	71	79	0	180
Total	459	93	205	143	18	441

A.6 Enforcement layers

Table 11. Representative invariants and failure boundaries in the implemented interaction path.

Layer	Representative invariant	Failure boundary
JSON schema	Required fields, types, enumerations, reference shapes	Before constructing a model instance
Canonical V2 model	Referential closure, typed cardinalities, source-ID uniqueness, assertion/validation ownership, no direct step-to-action shortcut	Instance rejected before materialization
Executable profile	One provider and capability per occurrence, provider/performer/target agreement, complete binding and action	Invalid chains are not published
Backend runtime	Pinned hash, unique trusted entity, derived group/zone, unique provider, local/direct route, exact target-specific action, request/response schema	Before generation and mutation when request-decidable; rollback after a response-dependent failure
Unity client	Exact collider binding, explicit unresolved states, returned target/provider/route/action agreement	Before request serialization or answer presentation; backend remains authoritative

B Artifact Map

The reproducible benchmark is rooted at `research/iui2027/evaluation`. Its `results.json` contains raw records; `tables.md` contains regenerated paper summaries; and `environment.json` records hashes and the local timing environment. The verifier and frozen regeneration entry points are under `research/iui2027/artifact`. Backend runtime code is under `InteractivAgents/openai_unity_expert_npcs_pycharm/InteractiveAgents/backend`, and Unity runtime components are under the corresponding `unity_scripts/FunctionalMldsV2` directory. The original manuscript remains in `research/iui2027/paper`; this reframed manuscript is deliberately separate.

References

[1] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, 1–13. doi:10.1145/3290605.3300233

[2] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. 2021. Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, 1–16. doi:10.1145/3411764.3445717

[3] Timothy Bickmore, Laura Pfeifer, and Daniel Schulman. 2011. Relational Agents Improve Engagement and Learning in Science Museum Visitors. In *Intelligent Virtual Agents*. Springer, 55–67. doi:10.1007/978-3-642-23974-8_7

[4] Kadir Burak Buldu, Süleyman Özdel, Ka Hei Carrie Lau, Mengdi Wang, Daniel Saad, Sofie Schönborn, Auxane Boch, Enkelejda Kasneci, and Efe Bozkir. 2025. CUIfy the XR: An Open-Source Package to Embed LLM-Powered Conversational Agents in XR. In *2025 IEEE International Conference on Artificial Intelligence and eXtended and Virtual Reality*. IEEE, 192–197. doi:10.1109/AIxVR63409.2025.00037

[5] Louis Burk, Alexander Fischer, Uwe Wienkop, Ramin Tavakoli Kolagari, Christoph Scharnagl, and Alexandra Arzberger. 2026. Handling Toolchain Evolution with Modular Meta-Languages: An Approach for Structured LLM-Based Artifact Generation. In *Proceedings of the 21st International Conference on Evaluation of Novel Approaches to Software Engineering*, Vol. 1. SciTePress, 807–814. doi:10.5220/0014715200004015

[6] Louis Burk and Uwe Wienkop. 2025. Dynamische Optimierung von VR-Lernumgebungen durch generative KI. In *Zukunft MINT Lehre: Was bleibt? Was kommt? Was wirkt? Tagungsband zum 6. Symposium zur Hochschullehre in den MINT-Fächern (Didaktiknachrichten)*, Hanna Dölling and Claudia Schäfle (Eds.). BayZiel – Bayerisches Zentrum für Innovative Lehre, München, 300–308. DiNa-Sonderausgabe, ISSN 1612-4537. https://bayziel.de/wp-content/uploads/Tagungsband_MINTSymposium_2025.pdf

[7] Gaëlle Calvary, Joëlle Coutaz, David Thevenin, Quentin Limbourg, Laurent Bouillon, and Jean Vanderdonckt. 2003. A Unifying Reference Framework for Multi-target User Interfaces. *Interacting with Computers* 15, 3 (2003), 289–308. doi:10.1016/S0953-5438(03)00010-9

[8] Rubén Campos-López, Esther Guerra, Juan de Lara, Alessandro Colantoni, and Antonio Garmendia. 2023. Model-Driven Engineering for Augmented Reality. *Journal of Object Technology* 22, 2 (2023), 2:1–2:15. doi:10.5381/jot.2023.22.2.a7

[9] Alessandro Carcangiu, Marco Manca, Jacopo Mereu, Carmen Santoro, Ludovica Simeoli, and Lucio Davide Spano. 2025. Tell-XR: Conversational End-User Development of XR Automations. In *Human-Computer Interaction – INTERACT 2025*. Springer, 617–641. doi:10.1007/978-3-032-04999-5_35

[10] Justine Cassell, Timothy W. Bickmore, Mark Billinghurst, Lee Campbell, Kenny Chang, Hannes Högni Vilhjálmsson, and Hao Yan. 1999. Embodiment in Conversational Interfaces: Rea. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 520–527. doi:10.1145/302979.303150

[11] Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. 2026. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 19888–19905. doi:10.18653/v1/2026.acl-long.912

[12] Fernanda De La Torre, Cathy Mengying Fang, Han Huang, Andrzej Banburski-Fahey, Judith Amores Fernandez, and Jaron Lanier. 2024. LLMR: Real-time Prompting of Interactive Worlds using Large Language Models. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, 1–22. doi:10.1145/3613904.3642579

[13] Anna Deichler, Jim O'Regan, Fethiye Irmak Dogan, Lubos Marcinek, Anna Klezovich, Iolanda Leite, and Jonas Beskow. 2026. MM-Conv: A Multimodal Dataset and Benchmark for Context-Aware Grounding in 3D Dialogue. arXiv:2605.21796 [cs.CV] doi:10.48550/arXiv.2605.21796

[14] EAST-ADL Association. 2013. *EAST-ADL Domain Model Specification*. Language Specification Version 2.1.12. EAST-ADL Association. https://east-adl.info/Specification/V2.1.12/EAST-ADL-Specification_V2.1.12.pdf

[15] Alexander Fischer, Louis Burk, Ramin Tavakoli Kolagari, and Uwe Wienkop. 2025. Machine-Readable by Design: Language Specifications as the Key to Integrating LLMs into Industrial Tools. In *Proceedings of the 20th Conference on Computer Science and Intelligence Systems*, Vol. 43. Polish Information Processing Society, 531–542. doi:10.15439/2025F5613

[16] Peter Gorniak and Deb Roy. 2005. Probabilistic Grounding of Situated Speech Using Plan Recognition and Reference Resolution. In *Proceedings of the 7th International Conference on Multimodal Interfaces*. ACM, 138–143. doi:10.1145/1088463.1088489

[17] Orlena C. Z. Gotel and Anthony C. W. Finkelstein. 1994. An Analysis of the Requirements Traceability Problem. In *Proceedings of the First IEEE International Conference on Requirements Engineering*. IEEE, 94–101. doi:10.1109/ICRE.1994.292398

[18] Sai Anirudh Karre and Y. Raghu Reddy. 2024. Model-based Approach for Specifying Requirements of Virtual Reality Software Products. *Frontiers in Virtual Reality* 5 (2024), 1471579. doi:10.3389/frvir.2024.1471579

[19] Casey Kennington and David Schlangen. 2015. Simple Learning and Compositional Application of Perceptually Grounded Word Meanings for Incremental Reference Resolution. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 292–301. doi:10.3115/v1/P15-1029

[20] Thomas Kollar, Stefanie Tellex, Deb Roy, and Nicholas Roy. 2010. Toward Understanding Natural Language Directions. In *2010 5th ACM/IEEE International Conference on Human-Robot Interaction*. IEEE, 259–266. doi:10.1109/HRI.2010.5453186

[21] Stefan Kopp, Lars Gesellensetter, Nicole C. Krämer, and Ipke Wachsmuth. 2005. A Conversational Agent as Museum Guide: Design and Evaluation of a Real-World Application. In *Intelligent Virtual Agents*. Springer, 329–343. doi:10.1007/11550617_28

[22] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. 2013. *PROV-O: The PROV Ontology*. W3C Recommendation. World Wide Web Consortium. https://www.w3.org/TR/2013/REC-prov-o-20130430/

[23] Séverin Lemaignan, Raquel Ros, E. Akin Sisbot, Rachid Alami, and Michael Beetz. 2012. Grounding the Interaction: Anchoring Situated Discourse in Everyday Human-Robot Interaction. *International Journal of Social Robotics* 4, 2 (2012), 181–199. doi:10.1007/s12369-011-0123-x

[24] Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *The Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303. doi:10.1016/j.jlap.2008.08.004

[25] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, Vol. 36. 51991–52008. doi:10.52202/075280-2264

[26] Q. Vera Liao, Daniel Gruen, and Sarah Miller. 2020. Questioning the AI: Informing Design Practices for Explainable AI User Experiences. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, 1–15. doi:10.1145/3313831.3376590

[27] Brian Y. Lim and Anind K. Dey. 2009. Assessing Demand for Intelligibility in Context-Aware Applications. In *Proceedings of the 11th International Conference on Ubiquitous Computing*. ACM, 195–204. doi:10.1145/1620545.1620576

[28] Quentin Limbourg, Jean Vanderdonckt, Benjamin Michotte, Laurent Bouillon, and Víctor López-Jaquero. 2005. USIXML: A Language Supporting Multi-path Development of User Interfaces. In *Engineering Human Computer Interaction and Interactive Systems*. Springer, 200–220. doi:10.1007/11431879_12

[29] Ziyi Liu, David Li, Zhongyi Zhou, David Kim, Ruofei Du, and Xun Qian. 2026. AgentHands: Generating Interactive Hand Gestures for Spatially Grounded Agent Conversations in XR. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. ACM, 1–24. doi:10.1145/3772318.3790938

[30] Patrick Mäder and Alexander Egyed. 2015. Do Developers Benefit from Requirements Traceability When Evolving and Maintaining a Software System? *Empirical Software Engineering* 20, 2 (2015), 413–441. doi:10.1007/s10664-014-9314-z

[31] Bertrand Meyer. 1992. Applying Design by Contract. *Computer* 25, 10 (1992), 40–51. doi:10.1109/2.161279

[32] Alex Moldovan, Vlad Nicula, Ionut Pasca, Mihai Popa, Jaya Krishna Namburu, Anamaria Oros, and Paul Brie. 2020. OpenUIDL, A User Interface Description Language for Runtime Omni-Channel User Interfaces. *Proceedings of the ACM on Human-Computer Interaction* 4, EICS (2020), 1–52. doi:10.1145/3397874

[33] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, 1–22. doi:10.1145/3586183.3606763

[34] Fabio Paternò, Carmen Santoro, and Lucio Davide Spano. 2009. MARIA: A Universal, Declarative, Multiple Abstraction-Level Language for Service-Oriented Applications in Ubiquitous Environments. *ACM Transactions on Computer-Human Interaction* 16, 4 (2009), 1–30. doi:10.1145/1614390.1614394

[35] Angel R. Puerta and Jacob Eisenstein. 1999. Towards a General Computational Framework for Model-Based Interface Development Systems. In *Proceedings of the 4th International Conference on Intelligent User Interfaces*. ACM, 171–178. doi:10.1145/291080.291108

[36] Justin D. Weisz, Jessica He, Michael Muller, Gabriela Hoefer, Rachel Miles, and Werner Geyer. 2024. Design Principles for Generative AI Applications. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, 1–22. doi:10.1145/3613904.3642466

[37] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. doi:10.1080/01621459.1927.10502953

[38] Qian Yang, Aaron Steinfeld, Carolyn Rosé, and John Zimmerman. 2020. Re-examining Whether, Why, and How Human-AI Interaction Is Uniquely Difficult to Design. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, 1–13. doi:10.1145/3313831.3376301